Day 26 🔥 — **Detect Loop in Linked List (Floyd Cycle Detection)**

🚀 1. Problem Samajh Lo

👉 Linked List me check karna hai ki cycle (loop) hai ya nahi

Example:

```
1 → 2 → 3 → 4 → 5
      ↑   ↓
      ← ← ←
```

👉 Yaha 5 wapas 3 ko point kar raha hai → Loop present

🧠 2. Approach (Floyd's Cycle Detection Algorithm)

Isko Tortoise and Hare Algorithm bhi bolte hain 🐢 🐇

Idea:

👉 2 pointers use karte hain:

Slow pointer → 1 step move karega

Fast pointer → 2 step move karega

🔥 Logic:

👉 Agar loop nahi hai:

Fast pointer null ho jayega

👉 Agar loop hai:

Slow aur Fast pointer kabhi na kabhi mil jayenge

📊 3. Visualization

Slow → 1 step
Fast → 2 steps

Step 1:
Slow = 2
Fast = 3

Step 2:
Slow = 3
Fast = 5

Step 3:
Slow = 4
Fast = 4 ✅ (Both meet)

👉 Meeting point mil gaya = Loop detected

🧪 4. Dry Run (Important for Interview)

List:

1 → 2 → 3 → 4 → 5 → 3 (loop)

Step	Slow	Fast
1	2	3
2	3	5
3	4	4 ✅

👉 Slow == Fast → Loop confirmed

💻 5. Java Code (Clean + Interview Ready)

```java
package Day_26;

public class DetectLoop {

    class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }

    Node head;

    // Insert at end
    public void insert(int data) {
        Node newNode = new Node(data);

        if (head == null) {
            head = newNode;
            return;
        }

        Node temp = head;
        while (temp.next != null) {
            temp = temp.next;
        }

        temp.next = newNode;
    }

    // Print list (safe print - stops if loop)
    public void printList() {
        Node temp = head;
        int count = 0;

        while (temp != null && count < 10) { // safety limit
            System.out.print(temp.data + " -> ");
            temp = temp.next;
            count++;
        }
        System.out.println("NULL");
    }

    // Detect Loop (Floyd Algorithm)
    public boolean hasLoop() {
        Node slow = head;
        Node fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;

            if (slow == fast) {
                return true;
            }
        }
        return false;
    }

    // Create Loop (for testing)
    public void createLoop() {
        if (head == null) return;

        Node temp = head;
```

```java
        Node thirdNode = null;
        int count = 1;

        while (temp.next != null) {
            if (count == 3) {
                thirdNode = temp;
            }
            temp = temp.next;
            count++;
        }

        temp.next = thirdNode; // last node → 3rd node
    }

    public static void main(String[] args) {
        DetectLoop list = new DetectLoop();

        list.insert(1);
        list.insert(2);
        list.insert(3);
        list.insert(4);
        list.insert(5);

        System.out.println("Linked List:");
        list.printList();

        // Create loop
        list.createLoop();

        // Detect loop
        if (list.hasLoop()) {
            System.out.println("Loop Detected");
        } else {
            System.out.println("No Loop ");
        }
    }
}

```

⚡ 6. Time & Space Complexity

Type	Complexity
Time	O(n)
Space	O(1)

👉 No extra memory → very efficient 💯